Detour

Walk the cool way.

Mathematical Modelling Guide

Every formula behind the backend, derived step by step and matched to the line of Python that computes it: coordinates, the sun, shadows, exposure integrals, graphs, A*, Pareto search and the avoidance slider.

L = h / tan θ
shadow length

∫ I (1 − f) dt
sun-seconds on a walkway

g + h
A* search key

Pareto
every time/sun trade-off

How to read this guide

- Each chapter follows the data through the backend in the order the code uses it: **where** → **when** → **shadow** → **exposure** → **graph** → **search** → **answer**.
- Every formula is followed by the code that computes it, with the file name above the snippet. Snippets are copied from the final code (comments may be shortened).
- Worked example boxes use real numbers produced by running the code. Model choice boxes say where the model simplifies reality and why.
- Chapter 14 is a one-page formula sheet for revision.

Contents

1 The model in one page
2 Coordinates and projection
3 Where is the sun?
4 Shadow geometry
5 Which walkways are in shade?
6 Sun and rain intensity
7 Exposure integrals
8 The campus as a graph
9 Shortest paths: Dijkstra and A*
10 Two objectives: the Pareto set
11 The slider: choosing one route
12 From routes to numbers on screen
13 Assumptions and error budget
14 Formula sheet

1 The model in one page

Question: for a walk across campus leaving at time t_0, how long does each possible route take, and how much direct sun does it get? Then: which routes are worth showing, and which one matches the user's slider?

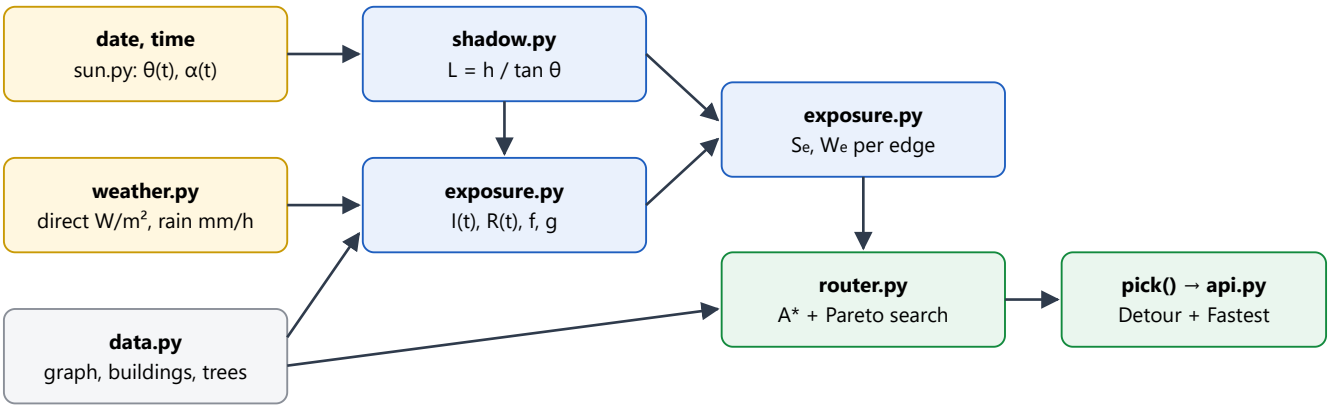


Figure 1. The modelling pipeline. Yellow = inputs from the world, blue = physics and geometry, green = optimisation.

1.1 Notation used throughout

Symbol	Meaning	Unit	In code
θ	sun elevation above the horizon	degrees	`sun.elevation`
α	sun azimuth, clockwise from north	degrees	`sun.azimuth`
h	height of a building or tree part	m	`height_m`
L	horizontal shadow length	m	`low`, `high`, `length`
e, d_e	a walkway edge and its length	m	`EdgeRef.length`
v	walking speed	1.4 m/s	`WALK_SPEED_MPS`
τ_e	time to walk edge e	s	`traversal_seconds()`
$f(e,t)$	shade fraction of edge e at time t	0–1	`shade_fractions()`
$g(e)$	shelter (roof) fraction of edge e	0–1	`shelter`
$I(t)$	relative direct-sun intensity	0–1 (≈)	`sun.intensity()`
$R(t)$	relative rain intensity	0–1	`sun.rain()`
S_e, W_e	sun-seconds and rain-seconds on edge e	s	`sun_seconds()`, `rain_seconds()`
w_r	rain weight (0, or 10 with "Avoid rain")	s per rain-s	`w_rain`
p	sun avoidance as a fraction	0–1	`sun_avoidance / 100`

THE TWO NUMBERS EVERY ROUTE GETS

A route P is a list of edges. Its **time** is $T(P) = \Sigma\, \tau_e$ and its **sun** is $S(P) = \Sigma\, S_e(t_e)$, where t_e is the moment you step onto edge e. Everything in chapters 2–7 exists to compute S_e; chapters 8–11 decide which routes to show.

2 Coordinates and projection

Latitude and longitude are angles on a sphere. Shadows and walking distances need metres. So all maths happens in a flat, metric map projection.

2.1 Why degrees are not metres

On the Earth (radius ≈ 6,371 km) one degree of latitude is always about 111.1 km. One degree of longitude shrinks towards the poles because the circles of latitude get smaller:

$$1° \text{ of longitude} \approx 111.32 \text{ km} \times \cos \varphi$$

> **WORKED EXAMPLE (CAMPUS, Φ = 43.7735°)**
>
> 111.32 × cos 43.7735° = **80.4 km** per degree of longitude, but 111.1 km per degree of latitude. Measuring distance directly in degrees would stretch north–south distances by 38 % compared with east–west ones. Shadows would point the wrong way and be the wrong length.

2.2 UTM zone 17N

The backend projects everything to **UTM zone 17N (EPSG:32617)**, a Transverse Mercator projection whose central meridian (81° W) passes close to Toronto. Within a zone, scale error is below 0.1 %, so over a 2 km campus x and y are true metres to within centimetres: x = metres east, y = metres north.

`backend/data.py and backend/api.py`

```
METRIC_CRS = "EPSG:32617"
_to_lonlat = Transformer.from_crs(METRIC_CRS, "EPSG:4326", always_xy=True)
_to_metric = Transformer.from_crs("EPSG:4326", METRIC_CRS, always_xy=True)
x, y = _to_metric.transform(lon, lat)   # always_xy: longitude (x) first, then latitude (y)
```

With the projection, distance is ordinary Euclidean geometry:

$$|AB| = \sqrt{(x_B - x_A)^2 + (y_B - y_A)^2}$$

That is exactly `math.hypot(dx, dy)`, used by the A* heuristic (chapter 9).

2.3 Rounding the output

Shadow polygons are sent back in degrees rounded to 6 decimals. How big is that error?

$$10^{-6}° \text{ of latitude} \times 111{,}132 \text{ m}/° \approx \mathbf{0.11 \text{ m}}$$

`backend/api.py, get_shadows()`

```
geometry = shapely.transform(_to_wgs84(shapely.multipolygons(pieces)), lambda c: np.round(c, 6))
```

A screen pixel at zoom 17 covers about 0.9 m on campus, so 0.11 m is invisible, while dropping digits makes the response about 40 % smaller.

2.4 Snapping a pin: nearest neighbour

A clicked point q becomes the graph node that minimises Euclidean distance:

$$\text{node}(q) = \text{argmin}_{v \in V} |q - v|$$

Checking all 1,804 nodes every time would work, but the backend builds an **STRtree** (a tree of bounding rectangles) once, so the search only looks at nearby rectangles: roughly $O(\log |V|)$ instead of $O(|V|)$.

backend/api.py

```
state.node_tree = shapely.STRtree(shapely.points([(nodes[n]["x"], nodes[n]["y"]) for n in
state.node_ids]))

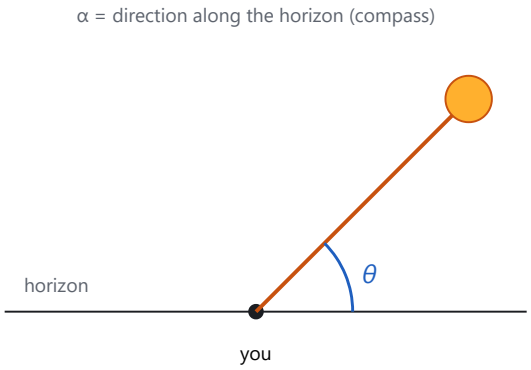
def _nearest_node(lat: float, lon: float) -> int:
    x, y = _to_metric.transform(lon, lat)
    return state.node_ids[int(state.node_tree.query_nearest(shapely.Point(x, y), all_matches=False)[0])]
```

Before snapping, `shapely.contains_xy(campus, x, y)` runs a point-in-polygon test (the same ray-casting idea the frontend uses): count how many polygon edges a ray from the point crosses; odd means inside.

3 Where is the sun?

3.1 Two angles describe the sun

- **Elevation θ**: angle above the horizon (0° = horizon, 90° = overhead).
- **Azimuth α**: compass bearing, clockwise from north (90° east, 180° south, 270° west).



The unit vector pointing *towards* the sun, in (east, north, up) coordinates, is

$$\hat{s} = (\cos\theta \sin\alpha,\ \cos\theta \cos\alpha,\ \sin\theta)$$

Check: α = 180° (south) gives (0, −cos θ, sin θ): pointing south and up. ✓

3.2 Where the angles come from (the textbook model)

The code uses **pvlib**'s implementation of the NREL Solar Position Algorithm, accurate to about 0.0003°. It is long, but its core is the classic spherical-astronomy relation, worth knowing for judges:

$$\delta \approx 23.44° \cdot \sin(\,360°/365 \cdot (284 + n)\,) \quad (\text{declination, } n = \text{day of year})$$

$$\sin\theta = \sin\varphi \cdot \sin\delta + \cos\varphi \cdot \cos\delta \cdot \cos H \quad (\varphi = \text{latitude, } H = \text{hour angle, } 15° \text{ per hour from solar noon})$$

At solar noon H = 0, and the formula reduces to $\theta_{noon} = 90° - \varphi + \delta$.

Day	δ	90° − φ + δ (by hand)	The code (pvlib)
21 June	+23.44°	90 − 43.77 + 23.44 = **69.67°**	69.67° at 13:20 EDT
20 March	0°	90 − 43.77 = **46.23°**	46.3° at 13:20 EDT
21 December	−23.44°	90 − 43.77 − 23.44 = **22.79°**	22.8° at 12:15 EST

> **WHY IS SOLAR NOON AT 13:20 IN JUNE?**
> Clocks in summer run on EDT (UTC−4), an hour ahead of standard time, and Toronto sits 4.5° west of the EST meridian (75° W), which adds about 18 minutes (4 min per degree). The equation of time shifts it by a few more minutes.

pvlib returns the **apparent** elevation, which includes atmospheric refraction (light bends in air, so the sun looks slightly higher, most noticeably near the horizon).

`backend/sun.py`

```
def sun_positions(lat, lon, times):
    sp = solarposition.get_solarposition(times, lat, lon)
    return sp[["apparent_elevation", "azimuth"]].rename(columns={"apparent_elevation": "elevation"})
```

3.3 A lookup table instead of recomputing

The router asks for the sun thousands of times per search, so `SunSchedule` computes all 24 × 60 + 1 = **1,441** minutes of a day once and then interpolates. For a time t that is m minutes after midnight (m can be fractional), with $i = \lfloor m \rfloor$:

$$\text{value}(t) = v_i + (v_{i+1} - v_i) \cdot (m - i)$$

This is linear interpolation: walk along the straight line between the two neighbouring minutes.

`backend/exposure.py, SunSchedule`

```
times = pd.to_datetime([self.t0 + timedelta(minutes=i) for i in range(24 * 60 + 1)], utc=True)
...
def _interp(self, values, when):
    m = min(max(self._minutes(when), 0.0), len(values) - 1.0)
    i = min(int(m), len(values) - 2)
    return float(values[i] + (values[i + 1] - values[i]) * (m - i))
```

The sun position itself (used for shadows) is taken at the **nearest** minute: `i = round(m)`. In one minute the sun rises or sinks at most about 0.18° on campus, and around a summer noon its azimuth turns about 0.66°.

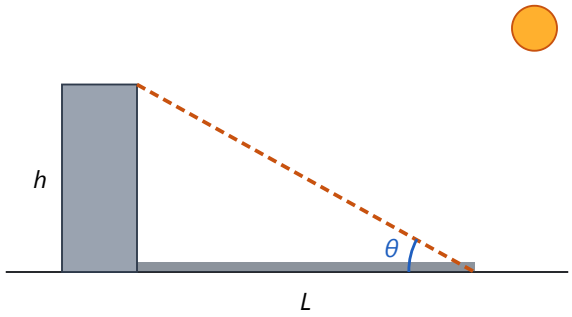
> **TIME ZONES**
> Times arrive as Toronto wall-clock time. The minute table is built in wall-clock minutes from local midnight and converted to UTC only for pvlib, so on the two daylight-saving days (23 h and 25 h days) the lookup still lines up.

4 Shadow geometry

4.1 Shadow of a single point

A point at height *h* blocks the sun ray that passes through it. That ray reaches the ground a horizontal distance *L* away, making a right-angled triangle with the vertical side *h*, the ground side *L* and the angle θ at the ground:

$$\tan\theta = \frac{\text{opposite}}{\text{adjacent}} = \frac{h}{L} \Longrightarrow L = \frac{h}{\tan\theta}$$



The shadow points **away** from the sun. The sun's horizontal direction is $(\sin\alpha, \cos\alpha)$ (east, north), so the shadow offset is the opposite vector scaled by L:

$$d = (\Delta x, \Delta y) = L \cdot (-\sin\alpha, -\cos\alpha)$$

Sun	α	(−sin α, −cos α)	Shadow points
due south	180°	(0, +1)	north ✓
due east (morning)	90°	(−1, 0)	west ✓
due west (evening)	270°	(+1, 0)	east ✓

`backend/shadow.py, ShadowCaster.pieces()`

```
if sun.elevation < MIN_ELEVATION_DEG:          # 3 degrees
    return np.empty(0, dtype=object)
tan_el = math.tan(math.radians(sun.elevation))  # trig needs radians: rad = deg × π / 180
alpha = math.radians(sun.azimuth)
direction = np.array([-math.sin(alpha), -math.cos(alpha)])
high = np.minimum(self._cvx_h / tan_el, MAX_SHADOW_LENGTH_M)   # L = h / tan θ, capped at 300 m
```

4.2 Why there is a 3° cut-off and a 300 m cap

As θ → 0, tan θ → 0 and L = h / tan θ → ∞. At θ = 3°, tan θ = 0.0524, so a 20 m building would cast a **382 m** shadow; any building taller than 300 × 0.0524 = **15.7 m** already hits the cap. Near sunrise and sunset the direct beam is also so weak (it passes through a lot of air) that shade barely matters, so the model stops there.

θ	tan θ	20 m building	61.5 m (tallest on campus)
69.6° (June noon)	2.69	7.4 m	22.9 m
46.3° (equinox noon)	1.05	19.1 m	58.8 m
22.8° (December noon)	0.420	47.6 m	146.4 m
3° (cut-off)	0.052	381.6 m → 300 m	→ 300 m

4.3 Shadow of a building: sweeping a prism

A building is modelled as a **prism**: its footprint polygon *P* extruded up to height *h*. A horizontal slice at height *z* (0 ≤ z ≤ h) casts a copy of *P* shifted by $(z / \tan\theta) \cdot u$, where u = (−sin α, −cos α). The whole shadow is the union of all those copies:

$$\text{Shadow}(P) = \bigcup_{0 \le s \le 1} (P + s \cdot d), \quad d = \frac{h}{\tan\theta} \cdot u$$

This "sweep of P along d" is called the **Minkowski sum** of P with the segment [0, d].

Convex footprints: one convex hull

If P is convex, the sweep equals the **convex hull** of P and its shifted copy P + d:

$$P \oplus [0, d] = \text{conv}(P \cup (P + d))$$

Why: every point of the sweep is p + s·d = (1 − s)·p + s·(p + d), a convex combination of a point of P and a point of P + d, so it lies in the hull. Conversely, any convex combination of points of P and P + d can be rewritten as (point of P) + s·d because P is convex. So both sets are equal.

The code treats a footprint as convex if it has no holes and fills at least 98 % of its own convex hull (`CONVEX_AREA_RATIO`), then computes all those hulls in one vectorised call:

backend/shadow.py

```
both = np.concatenate([
    self._cvx_xy + low[self._cvx_owner, None] * direction,    # every vertex shifted by the base offset
    self._cvx_xy + high[self._cvx_owner, None] * direction,   # ... and by the top offset
])
idx = np.concatenate([self._cvx_owner, self._cvx_owner])
order = np.argsort(idx, kind="stable")
pieces.append(shapely.convex_hull(shapely.multipoints(both[order], indices=idx[order])))
```

Concave footprints: one parallelogram per wall

For an L- or U-shaped building the hull would wrongly fill the courtyard. Instead each wall segment *[a, b]* sweeps a parallelogram with corners a, b, b + d, a + d, and

$$\text{Shadow}(P) = P \cup \bigcup_{\text{walls } [a,b]} \text{parallelogram}(a, b, b + d, a + d)$$

Why this covers everything: take any shadow point x = p + s·d. Walk from p towards x. Either you never leave P (so x ∈ P), or you cross a wall at some point b' and the rest of the way is a fraction of d, so x lies in that wall's parallelogram.

```
length = np.minimum(self._h / tan_el, MAX_SHADOW_LENGTH_M)
offset = length[:, None] * direction
ring = np.stack([self._p1, self._p2, self._p2 + offset, self._p1 + offset, self._p1], axis=1)
sweeps = shapely.polygons(ring)
pieces.append(self._ccv_fp)                          # the footprint itself
pieces.append(sweeps[shapely.area(sweeps) > 0])    # walls parallel to the sun sweep no area
```

4.4 Trees: a floating disk

Each tree is a crown: a disk of radius r = 3 m between heights b = 2 m (canopy base) and t = 9 m (top). Its slices from b to t give the convex sweep

$$\text{Shadow}(\text{tree}) = \text{conv}\left(D + (b / \tan\theta)\cdot u \;\cup\; D + (t / \tan\theta)\cdot u \right)$$

The disk is drawn as a 12-sided polygon (`quad_segs=3`), which has $(12 / 2\pi) \cdot \sin(2\pi/12) = 95.5\ \%$ of the circle's area.

> **DERIVED FACT: WHEN IS THE GROUND UNDER A TREE SHADED?**
> The point under the trunk is in shadow only if the lowest shadow disk still covers it, i.e. its centre is at most r away: $b / \tan\theta \le r \Leftrightarrow \tan\theta \ge 2/3 \Leftrightarrow$ **$\theta \ge 33.7°$**. Below that the shade falls beside the tree, exactly like real trees in the evening.

4.5 Drawing many overlapping pieces as one shape

Merging thousands of pieces into one polygon (a union) takes seconds per time step. Instead every piece is **oriented** (outer rings counter-clockwise) and the browser fills with the **nonzero winding rule**: a point is filled if the sum of signed windings around it is not zero. Each piece containing the point adds +1, so a point inside any piece is filled exactly once. The picture equals the union without computing it.

```
return shapely.orient_polygons(self._slice_pieces(when))   # exposure.py, shadow_pieces_at()
```

5 Which walkways are in shade?

5.1 The shade fraction

The quantity we need is the fraction of an edge's length that lies in shadow:

$$f(e, t) = \frac{\text{length}(\, e \cap \text{shadow}(t)\,)}{\text{length}(e)}$$

Computing that exactly for 5,172 edges against thousands of shadow polygons is slow, so the backend estimates it with **sample points**. Edge e of length ℓ gets

$$n_e = \max(\,2, \lceil\, \ell \,/\, 1.5\ \text{m}\,\rceil\,) \quad \text{points at positions} \quad (k + ½) \,/\, n_e \;\; (k = 0 \ldots n_e - 1) \text{ of the way along}$$

$$f(e, t) \approx \frac{1}{n_e} \, \Sigma_k \, \mathbb{1}[\text{ point } k \text{ is inside a shadow piece }]$$

This is the **midpoint rule** applied to the 0/1 "in shadow" function along the edge. Each shadow boundary that crosses the edge can be misplaced by at most half a spacing (0.75 m); in a whole-campus comparison at 9:00, 13:00 and 17:00 the length-weighted error was under 0.5 percentage points.

`backend/exposure.py, ExposureModel.__init__ (built once)`

```
counts = np.maximum(2, np.ceil(shapely.length(self._lines) / SAMPLE_SPACING_M)).astype(int)
self._sample_edge = np.repeat(np.arange(len(self._lines)), counts)            # which edge each point is
on
within = np.arange(counts.sum()) - np.repeat(np.cumsum(counts) - counts, counts)   # k for each point
frac = (within + 0.5) / np.repeat(counts, counts)                             # (k + 1/2) / n
self._samples = shapely.line_interpolate_point(self._lines[self._sample_edge], frac, normalized=True)
self._sample_tree = shapely.STRtree(self._samples)
```

Result: **121,632** fixed points. Per time step, one spatial query finds every point inside any piece, and `np.bincount` averages them per edge:

```
shaded[np.unique(self._sample_tree.query(pieces, predicate="intersects")[1])] = 1.0
frac = self._per_edge(shaded)   # bincount(edge, weights=shaded) / counts  =  (1/n) Σ 1[in shadow]
```

5.2 Time slices

Shadows move slowly, so they are computed once per 15-minute **slice** and reused. A time t (in seconds since 1970) belongs to slice

$$k = \text{round}(\, t \,/\, 900\,), \quad \text{slice time} = 900\, k$$

so t is never more than 7.5 minutes from its slice time. In 7.5 minutes the sun's elevation changes by at most about 1.4° (at 9:00 in June it rises 2.7° per 15 min), which moves a 20 m building's shadow edge by a metre or two at mid-morning.

```
def slice_index(self, when):
    return int(round(when.timestamp() / SLICE_SECONDS))   # SLICE_SECONDS = 900
```

Because slice k means the same instant for every date and weather, the shade fractions are cached by k and shared by all models (up to 192 slices = two days).

5.3 Shelter from rain

The shelter fraction g(e) uses the same sample points: the share inside a building footprint. If OpenStreetMap tags the walkway as roofed (`covered` , `indoor` , `tunnel` , `highway=corridor`), g is set to 1.

$$g(e) = \frac{1}{n_e} \Sigma_k \mathbb{1}[\text{ point k inside a building }] \quad \text{(or 1 if tagged as roofed)}$$

364 directed edges have g > 0.5. g does not change with time, so it is computed once at start-up.

6 Sun and rain intensity

6.1 What shade actually blocks

Sunlight reaching the ground has two parts: the **direct beam** from the sun's disk, and **diffuse** light scattered by the sky and clouds. A shadow removes only the direct beam; diffuse light reaches you in the shade too. So the model's sun intensity is based on direct radiation only.

$$I(t) = \frac{\text{direct radiation}(t)\ [\mathrm{W/m^2}]}{1000\ \mathrm{W/m^2}}$$

1000 W/m² is roughly the strongest direct beam at the ground on a clear day, so I ≈ 1 is "full sun" and I can very slightly exceed 1.

6.2 Clear-sky fallback: Lambert's cosine law

When no weather is available, the model assumes a clear sky with $I(t) = \max(0, \sin\theta(t))$. The reason is geometry: a beam of cross-section A hitting the flat ground at elevation θ spreads over an area A / sin θ, so the power per square metre of ground is multiplied by sin θ (Lambert's cosine law, with the zenith angle 90° − θ). Below the horizon there is no direct sun, hence the max(0, ·).

`backend/exposure.py, SunSchedule.__init__`

```
if weather is None:
    self._intensity = np.maximum(0.0, np.sin(np.radians(self.elevation)))
    self._rain = np.zeros(len(times))
else:
    self._intensity = weather.direct / CLEAR_SKY_DIRECT_WM2   # 1000 W/m²
    self._rain = rain_intensity(weather.rain)
```

MODEL CHOICE

The fallback ignores the atmosphere's absorption (a low sun passes through more air and is dimmer than sin θ suggests). It only applies when live data is missing; with live weather the measured/forecast direct radiation already includes clouds and air mass.

6.3 Turning hourly weather into minutes

Open-Meteo gives one value per hour. Radiation and precipitation are **averages over the hour ending** at the timestamp, so each value is placed at the middle of its hour (−30 min) and the 1,441-minute table is filled by linear interpolation between those points:

$$y(m) = y_j + (y_{j+1} - y_j) \cdot \frac{m - m_j}{m_{j+1} - m_j}, \quad m_j = 60\,j - 30$$

backend/weather.py, DayWeather.from_hourly()

```
minutes = np.arange(24 * 60 + 1)
at_hour = np.arange(24) * 60.0
mid_hour = at_hour - 30.0  # radiation and precipitation are averages over the preceding hour
return cls(
    direct=np.interp(minutes, mid_hour, values["direct_radiation"]),
    cloud=np.interp(minutes, at_hour, values["cloud_cover"] / 100.0),
    rain=np.interp(minutes, mid_hour, values["precipitation"]),
)
```

Cloud cover is an instantaneous value, so it stays at the hour; it is only shown to the user. Clouds still change the route, through lower direct radiation.

6.4 Rain intensity

$$R(t) = \min\left(1,\ \frac{\text{rain}(t)\ [\text{mm/h}]}{4\ \text{mm/h}}\right)$$

4 mm/h is "moderate rain"; anything heavier counts as fully wet (R = 1). Light drizzle of 1 mm/h gives R = 0.25.

```
def rain_intensity(rain_mm_h):
    return np.minimum(1.0, np.asarray(rain_mm_h) / 4.0)
```

7 Exposure integrals

7.1 Time on an edge

$$\tau_e = \frac{d_e}{v}, \quad v = 1.4 \text{ m/s } (\approx 5 \text{ km/h})$$

```
def traversal_seconds(self, edge):
    return self.edges[edge].length / WALK_SPEED_MPS
```

7.2 Sun-seconds: an integral over the walk

If you step onto edge e at time t_e, you are on it from t_e to $t_e + \tau_e$. At each instant you receive sun intensity I(t), except on the shaded fraction f of the edge. Adding it up over time:

$$S_e(t_e) = \int_{t_e}^{t_e + \tau_e} I(t) \cdot (1 - f)\, dt$$

The unit is **sun-seconds**: one second in full direct sun. Two seconds at half intensity is also one sun-second. f is taken from the time slice at the moment you enter the edge (edges take seconds, slices last 15 minutes).

7.3 The trapezoid rule

The integral is approximated by the average of the start and end values times the duration:

$$\int_a^b I(t)\, dt \approx (b - a) \cdot \frac{I(a) + I(b)}{2}$$

$$S_e \approx \tau_e \cdot \frac{I(t_e) + I(t_e + \tau_e)}{2} \cdot (1 - f)$$

backend/exposure.py

```
def _trapezoid(self, edge, enter, rate):
    tau = self.traversal_seconds(edge)
    return tau * 0.5 * (rate(enter) + rate(enter + timedelta(seconds=tau)))

def sun_seconds(self, edge, enter):
    return (1.0 - self.shade_fractions(enter)[edge]) * self.full_sun_seconds(edge, enter)

def full_sun_seconds(self, edge, enter):          # same integral with f = 0
    return self._trapezoid(edge, enter, self.sun.intensity)
```

HOW ACCURATE IS THE TRAPEZOID RULE HERE?

The error is $-(b-a)^3 / 12 \cdot I''(\xi)$ for some ξ in the interval. Clear-sky intensity varies with the Earth's rotation, ω = 2π / 86,400 s ≈ 7.3×10^{-5} rad/s, so $|I''| \le \omega^2 \approx 5.3 \times 10^{-9}$ s^{-2}. For a 60-second edge the error is at most $60^3 / 12 \times 5.3 \times 10^{-9} \approx$ **0.0001 sun-seconds**. With live weather, I is piecewise linear in minutes, and the trapezoid rule is exact on every straight piece.

WORKED EXAMPLE

A 14 m edge: τ = 14 / 1.4 = 10 s. I = 0.80 when you enter, 0.82 when you leave; f = 0.25.
S = 10 × (0.80 + 0.82) / 2 × (1 − 0.25) = 10 × 0.81 × 0.75 = **6.08 sun-seconds**. With no shade it would be 8.10.

7.4 Rain-seconds

Exactly the same idea, with rain intensity and shelter instead of sun and shade:

$$W_e(t_e) = \int R(t) \cdot (1 - g(e))\, dt \approx \tau_e \cdot \frac{R(t_e) + R(t_e + \tau_e)}{2} \cdot (1 - g(e))$$

```
def rain_seconds(self, edge, enter):
    return (1.0 - self.shelter[edge]) * self.full_rain_seconds(edge, enter)
```

7.5 Adding up a route

For a route of edges $e_1 \dots e_k$ leaving at t_0, each edge is entered when the previous ones are done:

$$t_{e_1} = t_0, \quad t_{e_{j+1}} = t_{e_j} + \tau_{e_j} \quad T = \Sigma\, \tau_e \quad S = \Sigma\, S_e(t_e) \quad S_{open} = \Sigma\, S_e \text{ with } f = 0$$

$$\text{sun \%} = 100 \cdot S \,/\, S_{open} \quad (0 \text{ if } S_{open} = 0)$$

backend/router.py, _summarise()

```
t = 0.0
for ei in edges:
    enter = start + timedelta(seconds=t)
    sun += model.sun_seconds(ei, enter)
    full_sun += model.full_sun_seconds(ei, enter)
    rain += model.rain_seconds(ei, enter)
    full_rain += model.full_rain_seconds(ei, enter)
    dist += model.edges[ei].length
    t += model.traversal_seconds(ei)
```

WHAT SUN % REALLY MEASURES

Sun % compares the route with *the same route if nothing cast a shadow*. If the intensity were constant during the walk it would cancel out, and sun % = time-weighted share of the walk spent unshaded. It measures shade, not weather: a cloudy day lowers S and S_{open} together. Example: the demo trip's fastest route has S = 961.3 and sun % = 88.5 %, so S_{open} = 961.3 / 0.885 ≈ 1,086 sun-seconds.

8 The campus as a graph

$$G = (V, E), \quad |V| = 1{,}804 \text{ nodes}, \quad |E| = 5{,}172 \text{ directed edges}$$

- **Nodes** are walkway junctions and dead ends, each with metric coordinates (x, y).
- **Edges** are walkway segments. Every walkway appears twice (u → v and v → u), because exposure depends on which end you start from and when.
- It is a **multigraph**: two nodes can be joined by several different paths, told apart by a key (0, 1, …). An edge is identified by (u, v, key).
- It is **strongly connected**: from every node you can reach every other. That is why any two on-campus pins always have a route.

`backend/exposure.py: numbering the edges`

```
for u, v, k, a in data.graph.edges(keys=True, data=True):
    self.edge_index[(u, v, k)] = len(self.edges)
    self.edges.append(EdgeRef(u, v, float(a["length"])))
```

8.1 Edge cost

The router minimises a cost per edge that depends on when you step onto it:

$$c_e(t) = \tau_e + w_r \cdot W_e(t)$$

Both terms are seconds, so w_r reads as "how many seconds of extra walking is one rain-second worth?" With "Avoid rain" off, $w_r = 0$ and the cost is pure walking time. With it on, $w_r = 10$.

> **WORKED EXAMPLE: AVOIDING RAIN**
>
> 2 mm/h rain → R = 0.5. Open 30 s edge: W = 30 × 0.5 = 15 rain-seconds, cost = 30 + 10 × 15 = **180**. A covered 45 s corridor (g = 1): W = 0, cost = **45**. The corridor wins even though it is 15 s longer.

Sun is **not** part of this cost. It is kept as a separate second objective (chapter 10).

8.2 Time-dependent costs

Because S_e and W_e depend on the entry time, this is a **time-dependent shortest-path problem**. Each search label carries its own arrival time: $t_v = t_u + \tau_{uv}$. Settling nodes in cost order (as Dijkstra does) is exact if arriving earlier never makes the rest of the walk costlier (the "FIFO" property). Over a walk of minutes the weather and sun change very little, so the error is negligible.

9 Shortest paths: Dijkstra and A*

9.1 Dijkstra's algorithm

Keep a priority queue (a binary heap) of nodes ordered by $g(n)$, the best cost found so far from the origin. Repeatedly remove the cheapest node u and **relax** each outgoing edge:

$$\text{if } g(u) + c(u, v) < g(v): \quad g(v) \leftarrow g(u) + c(u, v), \quad \text{parent}(v) \leftarrow u$$

With non-negative costs, the first time a node is removed its g is optimal. Following parents back from the destination gives the route. Cost: $O((|V| + |E|) \log |V|)$ with a binary heap.

9.2 A*: add a lower bound

A* orders the heap by $f(n) = g(n) + h(n)$, where h(n) never overestimates the remaining cost. Detour uses straight-line walking time with a small slack:

$$h(n) = 0.99 \cdot \frac{|n - \text{goal}|}{1.4 \text{ m/s}}$$

backend/router.py

```
def _heuristic(model, dest):
    nodes = model.graph.nodes
    gx, gy = nodes[dest]["x"], nodes[dest]["y"]
    scale = HEURISTIC_SLACK / WALK_SPEED_MPS          # 0.99 / 1.4
    return lambda n: scale * math.hypot(nodes[n]["x"] - gx, nodes[n]["y"] - gy)
```

9.3 Proof that h is consistent

A heuristic is **consistent** if $h(u) \le c(u, v) + h(v)$ for every edge. Then A* settles exactly the nodes Dijkstra would, in a better order, and returns the same optimal cost.

1. Every edge costs at least its walking time, since the rain term is ≥ 0: $c(u, v) \ge d_{uv} / 1.4$.
2. On the campus graph every edge is at least 0.9975 times the straight line between its ends (OSM lengths can be a hair short), so $d_{uv} \ge 0.9975\,|uv| \ge 0.99\,|uv|$.
3. Triangle inequality: $|u - \text{goal}| \le |uv| + |v - \text{goal}|$.

$$h(u) = \frac{0.99\,|u - \text{goal}|}{1.4} \le \frac{0.99\,|uv|}{1.4} + \frac{0.99\,|v - \text{goal}|}{1.4} \le c(u, v) + h(v) \quad \checkmark$$

That is why the slack is 0.99 and not 1: with 1.0, step 2 could fail by a fraction of a percent. Measured benefit on the demo trip: A* settled **876** nodes, Dijkstra **1,703**.

9.4 The search loop in code

```
heap = [(heuristic(origin), 0.0, next(counter), origin)]       # (f, g, tie-breaker, node)
while heap:
    _, cost_u, _, u = heapq.heappop(heap)
    if cost_u != best.get(u):          # an outdated entry: a cheaper one was pushed later
        continue
    if u == dest:
        break
    t_u = start + timedelta(seconds=elapsed[u])                # arrival time at u
    for v, parallel in g[u].items():
        for key in parallel:
            ei = model.edge_index[(u, v, key)]
            tau = model.traversal_seconds(ei)
            c = best[u] + tau
            if w_rain:
                c += w_rain * model.rain_seconds(ei, t_u)
            if c < best.get(v, math.inf):                      # relax
                best[v] = c
                came[v] = (u, ei)
                elapsed[v] = elapsed[u] + tau
                heapq.heappush(heap, (c + heuristic(v), c, next(counter), v))
```

Instead of a "decrease key" operation, a node is simply pushed again with its new cost; old entries are skipped when popped (the `cost_u != best.get(u)` check). The counter breaks ties so Python never has to compare two nodes.

10 Two objectives: the Pareto set

10.1 Dominance

Each route has two numbers to make small: cost C (walking time, plus rain if weighted) and sun S. Route A **dominates** route B if it is at least as good on both and strictly better on one:

$$A \prec B \iff C_A \le C_B,\ \ S_A \le S_B,\ \ \text{and at least one strictly}$$

The **Pareto set** is all routes no other route dominates. Sorted by cost, its sun values strictly decrease: each option costs more and gets less sun than the one before. Every sensible "time for shade" trade lies in this set.

10.2 Why not one weighted cost?

The obvious idea is to minimise $C + w_s \cdot S$ for a sun weight w_s. Geometrically, that slides a straight line of slope $-1/w_s$ across the (C, S) plot until it touches a route. It can only touch routes on the **lower convex hull**. A route in a "dent" of the trade-off curve is never chosen for any w_s.

EXAMPLE

Three routes (C, S): A = (100, 100), B = (110, 85), D = (120, 40). B is Pareto-optimal (nothing beats it on both). But for any w_s:

A: 100 + 100 w, B: 110 + 85 w, D: 120 + 40 w.

B beats A only if w > 0.67, and beats D only if w < 0.22. Impossible at once, so a weighted sum **never** picks B, and the route jumps straight from A to D.

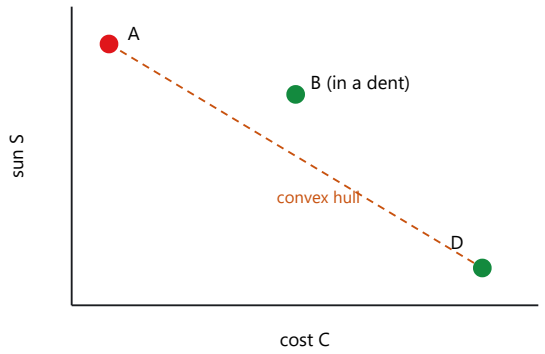


Figure 2. B lies above the hull line A–D, so no weight selects it.

10.3 Martins' label-setting algorithm

To find the whole set, the search keeps **labels** instead of one best value per node. A label is a partial route ending at a node: (cost so far, sun so far, time walked, parent label). Labels are popped in order of $cost + h(node)$ (A* keys again). At each node the algorithm remembers the least sun of any label kept there.

- A label is **kept** only if its sun is lower than every label kept at that node so far. Since labels arrive in cost order, a kept label is cheaper than later ones and shadier than earlier ones: never dominated.
- It is also discarded if it has more sun than an option already found at the destination (that option is cheaper and shadier).
- Every label reaching the destination is a new Pareto option.

backend/router.py, shade_options()

```
while heap:
    _, cost, sun, elapsed, _, u, parent, edge = heapq.heappop(heap)
    # dominated by a label kept here, or by an option already found (which is cheaper)
    if sun > least_sun.get(u, math.inf) - step or sun > least_sun.get(dest, math.inf) - step:
        continue
    least_sun[u] = sun
    labels.append((parent, edge))
    if u == dest:
        found.append((len(labels) - 1, cost))
        continue
    ...
            c = cost + tau + (w_rain * model.rain_seconds(ei, t_u) if w_rain else 0.0)
            if c + h(v) > cap:                # even a perfect straight walk would exceed the cap
                continue
            s = sun + model.sun_seconds(ei, t_u)
            if s <= least_sun.get(v, math.inf) - step:
                heapq.heappush(heap, (c + h(v), c, s, elapsed + tau, next(counter), v, len(labels) - 1,
ei))
```

10.4 Two limits keep the set small and useful

$$\text{cap} = C_{\text{quickest}} \cdot (1 + 0.5) \qquad \text{step} = \max(1,\ 0.02 \cdot S_{\text{quickest}})$$

- **Cost cap** (`MAX_DETOUR = 0.5`): no option may cost more than 1.5 × the quickest. Pruning with $c + h(v) > cap$ is safe because h is a lower bound: if even the straight-line remainder exceeds the cap, every real continuation does too.
- **Sun step** (`SUN_STEP = 0.02`): a new label must beat the previous least sun by at least 2 % of the quickest route's sun (and 1 sun-second). This is **ε-dominance**: routes that differ by a sliver of shade are treated as the same, so the list has a handful of meaningfully different routes instead of hundreds.

WORKED EXAMPLE: THE DEMO TRIP (21 JUNE 14:00, CLEAR SKY)

Option	Time (s)	Sun-s	Sun %
1 (quickest)	1177.7	961.3	88.5
2	1182.2	927.2	85.1
3	1188.7	896.6	81.8
4	1189.1	856.0	78.1
5	1219.2	827.7	73.7
6	1225.7	797.1	70.6
7	1226.1	756.5	67.0
8	1310.7	725.6	60.1
9 (shadiest)	1477.9	690.5	50.8

Here step = max(1, 0.02 × 961.3) = 19.2 sun-seconds and cap = 1.5 × 1177.7 = 1766.6 s. Each option is slower and shadier than the one above.

11 The slider: choosing one route

The slider value p ∈ [0, 1] (the UI's 0–100 % divided by 100) means "avoid at least a fraction p of the sun that *can* be avoided on this trip". With S_1 the quickest option's sun and S_k the shadiest's:

$$\text{target} = S_1 - p \cdot (S_1 - S_k)$$

$$\text{Detour} = \text{the first (quickest) option with } S \le \text{target}$$

backend/router.py

```
def pick(options, avoidance):
    most, least = options[0].sun_seconds, options[-1].sun_seconds
    target = most - avoidance * (most - least)
    return next(r for r in options if r.sun_seconds <= target + 1e-9)
```

- p = 0 → target = S_1 → the quickest option. p = 1 → target = S_k → the shadiest.
- As p grows the target falls, so the chosen route can only get shadier and slower: the slider is **monotonic**.
- The `1e-9` absorbs floating-point rounding so that p = 1 always finds the last option.

WORKED EXAMPLE: DEMO TRIP AT 50 %

target = 961.3 − 0.5 × (961.3 − 690.5) = 961.3 − 135.4 = **825.9**. Options 1–5 have more sun than that; option 6 (797.1) is the first below. Result: **1225.7 s** instead of 1177.7 s (+48 s), sun 88.5 % → 70.6 %.

WHY THIS IS FAST

The options depend on the trip, time and rain weight, not on p. They are cached, so moving the slider only re-runs `pick()` : a few comparisons on a list of about 1–12 routes.

11.1 The Fastest route

The red "Fastest" line is always `route(model, origin, dest, when)` with $w_r = 0$: pure walking time, whatever the settings. The Detour route comes from `pick()`. The difference shown in the panel is

$$\text{extra_seconds} = T_{\text{Detour}} - T_{\text{Fastest}}, \quad \text{sun_percent_change} = \text{sun }\%_{\text{Detour}} - \text{sun }\%_{\text{Fastest}} \text{ (percentage points)}$$

12 From routes to numbers on screen

12.1 Rounding and units in the response

Field	Formula	Rounding	Unit
`seconds`	$T = \Sigma\ \tau_e$	0.1 s	s
`distance_m`	$\Sigma\ d_e$	0.1 m	m
`sun_percent`	$100 \cdot S / S_{open}$	0.1	% (already ×100)
`rain_percent`	$100 \cdot W / W_{open}$	0.1	%
`sun.elevation_deg`	θ at the departure minute	0.01°	degrees
`weather.cloud_cover`	Open-Meteo cloud / 100	0.01	fraction 0–1

The panel rounds percentages to whole numbers (`Math.round`) and formats seconds as "8 min 18 sec".

12.2 Building the line on the map

Each edge has a curved geometry. Its points must run from the edge's start node u to its end node v; if the stored line runs the other way it is reversed. The test is a distance comparison:

$$\text{reverse if} \quad |\text{first point} - u| > |\text{last point} - u|$$

```
(x0, y0), (x1, y1) = line.coords[0], line.coords[-1]
if math.hypot(x0 - nu["x"], y0 - nu["y"]) > math.hypot(x1 - nu["x"], y1 - nu["y"]):
    line = LineString(list(line.coords)[::-1])
```

The edges' points are concatenated (dropping each repeated junction point), projected back to longitude/latitude and sent as a GeoJSON LineString.

12.3 Memory: a small calculation

A midday `/shadows` response is about 1.45 MB. The backend keeps at most 64 of them: 64 × 1.45 MB ≈ **93 MB**. On a 512 MB server with about 280 MB used by the loaded campus, that leaves a comfortable margin (192 cached responses would take about 280 MB on their own and could run the server out of memory).

13 Assumptions and error budget

Every model simplifies. This table lists each simplification, the size of its effect, and why it was accepted.

Assumption	Effect	Why accepted
Flat ground	Shadows on slopes are slightly longer or shorter	Keele campus is nearly flat
Buildings are prisms at maximum height	Overestimates shadows of buildings with lower wings	Toronto data gives one height per footprint
Uniform trees (3 m crown, 2–9 m)	Real trees vary; leaves assumed all year	OSM has no tree sizes
1.5 m sampling	$<$ 0.5 percentage points of shade (length-weighted)	Fast: one spatial query per slice
15-min time slices	Sun position off by $\leq$ 7.5 min ($\leq$ about 1.4° elevation)	Shadows move slowly; huge speed-up
Trapezoid rule	$\approx 10^{-4}$ sun-s per edge	Edges take seconds
Constant speed 1.4 m/s	No stairs, crowds or waiting	Standard average walking speed
No shade below 3°; 300 m cap	Ignores very long dawn/dusk shadows	Weak beam; $\tan\theta \to 0$
Clear-sky fallback $\sin\theta$	Overestimates sun near the horizon	Only used without weather data
One weather point, hourly	No local clouds across campus	Campus is 2 km wide
Time-dependent search settles by cost	Exact only if leaving earlier never hurts	Conditions change little within a walk
ε-dominance (2 % step)	Near-duplicate routes are merged	A short, meaningful list
Pin $\to$ nearest node	The dotted gap is not counted in time/distance	Pins can be anywhere on campus

IF A JUDGE ASKS "HOW ACCURATE IS IT?"

Name the largest effects honestly: building heights (maximum height per footprint), uniform trees and one weather point. The numerical methods (sampling, slices, trapezoid rule) contribute well under one percentage point.

14 Formula sheet

Quantity	Formula	Code
Degrees of longitude	$1° \approx 111.32 \text{ km} \cdot \cos\varphi$ (80.4 km on campus)	avoided by UTM 17N
Noon sun elevation	$\theta = 90° - \varphi + \delta$	pvlib (sun.py)
Linear interpolation	$v_i + (v_{i+1} - v_i)(m - i)$	`SunSchedule._interp`
Shadow length	$L = h / \tan\theta, \quad \le 300$ m, none if $\theta < 3°$	`ShadowCaster.pieces`
Shadow direction	$d = L\,(-\sin\alpha, -\cos\alpha)$	`direction`
Convex building shadow	$\text{conv}(P \cup (P + d))$	`convex_hull(multipoints)`
Concave building shadow	$P \cup \text{parallelograms}(a, b, b + d, a + d)$	`sweeps`
Tree shadow	$\text{conv}(D + (2/\tan\theta)\,u \cup D + (9/\tan\theta)\,u)$, $r = 3$ m	tree crowns in `convex`
Shade fraction	$f = (1/n) \sum \mathbb{1}[\text{sample in shadow}]$, $n = \max(2, \lceil \ell/1.5 \rceil)$	`shade_fractions`
Time slice	$k = \text{round}(t / 900\text{ s})$	`slice_index`
Sun intensity	$I = \text{direct} / 1000\ \text{W/m}^2$ (or $\max(0, \sin\theta)$)	`SunSchedule`
Rain intensity	$R = \min(1, \text{rain} / 4\ \text{mm/h})$	`rain_intensity`
Walking time	$\tau = d / 1.4$	`traversal_seconds`
Sun-seconds	$S_e \approx \tau \cdot (I(t_e) + I(t_e + \tau))/2 \cdot (1 - f)$	`sun_seconds`
Rain-seconds	$W_e \approx \tau \cdot (R(t_e) + R(t_e + \tau))/2 \cdot (1 - g)$	`rain_seconds`
Sun %	$100 \cdot S / S_{\text{open}}$	`RouteResult.sun_percent`
Edge cost	$c = \tau + w_r W, \quad w_r \in \{0, 10\}$	`route`
A* heuristic	$h(n) = 0.99 \cdot \lvert n - \text{goal} \rvert / 1.4$	`_heuristic`
Consistency	$h(u) \le c(u, v) + h(v)$	proof in §9.3
Pareto limits	$\text{cap} = 1.5\, C_{\text{quickest}}$; $\text{step} = \max(1, 0.02\, S_{\text{quickest}})$	`shade_options`
Slider target	$S_1 - p\,(S_1 - S_k)$; first option with $S \le \text{target}$	`pick`

THE WHOLE MODEL IN THREE LINES

1. Shadows: $L = h / \tan\theta$, swept from every building and tree, sampled every 1.5 m along every walkway.
2. Exposure: $S_e = \int I(t)(1 - f)\,dt$, with I from direct radiation, summed along the route at the time you reach each edge.
3. Choice: find every Pareto trade-off between cost and sun (A* keys, 50 % cap, 2 % step), then take the quickest that removes the chosen share of the avoidable sun.